РАЗДЕЛ 1 · ПРАКТИЧЕСКИЕ ЗАНЯТИЯ 3–4

Use Case и пользовательские сценарии C4 Context Diagram

Кейс 14: «Контроль заявок курьеров (SLA)» — актёры, роли и внешние системы на примере 1С

Где мы находимся в курсе

БЫЛО

Практики 1–2

Анализ рынка и аналогов → Vision продукта: границы системы и целевая аудитория зафиксированы

СЕГОДНЯ

Практики 3–4

От vision — к акторам и сценариям (Use Case), затем к первой архитектурной диаграмме (C4 Context)

ДАЛЬШЕ

Практики 5–6

C4 Container Diagram → C4 Component Diagram: раскрываем систему изнутри

План двух занятий

1. **Занятие 3 · Что такое Use Case**
   акторы, границы системы, нотация
2. **Занятие 3 · Анатомия сценария**
   предусловия, основной и альтернативные потоки
3. **Занятие 3 · Пример и проверка полноты**
   разбор кейса + чек-лист
4. **Занятие 3 · Практика: свои Use Case**
   актёры и сценарии для своего продукта
5. **Занятие 4 · От Use Case к Context Diagram**
   акторы → persons, зависимости → внешние системы
6. **Занятие 4 · Нотация C4 Context**
   элементы и подписанные связи
7. **Занятие 4 · Пример и проверка полноты**
   разбор кейса + чек-лист
8. **Занятие 4 · Практика: свой Context Diagram**
   рисуем диаграмму для своего продукта

СВЯЗЬ С ЛЕКЦИЕЙ 2

От НФТ кейса — к 5 атрибутам качества

Цитата из карточки кейса 14: «Производительность: расчёт SLA по ≥100 заказам и ≥10 рейсам — интерактивно. Надёжность: импорт CSV идемпотентен. Трассируемость: лог задач расчёта».

1	**Производительность**	✓ есть в кейсе	интерактивный расчёт SLA → предложите порог: APDEX ≥ 0,85, отклик ≤ 2 сек
2	**Масштабируемость**	⚠ не зафиксирована	в карточке нет — предложите сами: рост в 10× объёма заказов без деградации
3	**Надёжность**	✓ есть в кейсе	идемпотентный импорт CSV → 0 дублей при повторной загрузке
4	**Доступность**	⚠ не зафиксирована	в карточке нет — предложите сами: отчёт SLA доступен в рабочие часы координатора
5	**Сопровождаемость**	◐ косвенно	«трассируемость» лога расчёта помогает разбирать инциденты — это скорее про неё

Важный нюанс: «трассируемость» здесь — это лог выполнения расчёта (кто/когда/сколько нарушений), а не та трассируемость R→A из лекции 1, о которой на следующем слайде.

СВЯЗЬ С ЛЕКЦИЕЙ 1

Тест на архитектурное решение + трассируемость R→A

Решение: «Импорт CSV должен быть идемпотентным» — применим тест из лекции 1

- ✓ Меняет структуру? Да — нужна проверка дублей перед записью
- ✓ Влияет на несколько ролей? Да — администратор доверяет результату, координатор видит корректные метрики
- ✓ Влияет на качества системы? Да — это прямо надёжность из лекции 2
- ✓ Дорого поменять потом? Да — переделка логики импорта затронет уже загруженные данные
- ✓ Требует выбора между альтернативами? Да — можно было выбрать «запретить повторный импорт» вместо идемпотентности

Вывод: все 5 «да» — это архитектурное решение. Фиксируем трассируемость так же, как в лекции 1:

R-14-1 Надёжность: импорт CSV идемпотентен, без дублей заказов/статусов → **A-14-1** Use Case «Импортировать данные заказов»: альтернативный поток проверки дублей по order_id

В своей практике оформляйте так же: каждое архитектурно значимое решение в Use Case — с номером требования и коротким обоснованием.

ЗАНЯТИЕ 3

Что такое Use Case

Use Case описывает один законченный сценарий взаимодействия актёра с системой, который приносит актёру измеримый результат.



Актор — снаружи границы (человек или другая система). Use Case — внутри границы (то, что делает конфигурация 1С).

ЗАНЯТИЕ 3

Акторы: откуда они берутся

ПЕРВИЧНЫЙ АКТОР

инициирует сценарий ради своей цели — например, администратор, который импортирует данные о заказах и рейсах

ВТОРИЧНЫЙ АКТОР

обслуживает сценарий, но не инициирует его — например, мобильное приложение курьера, поставляющее статусы доставки

СВЯЗЬ С VISION И РОЛЯМИ 1С

Акторы не придумываются заново — они берутся из целевой аудитории, описанной в vision, и из внешних систем, упомянутых в границах продукта.

В 1С у каждого первичного актёра обычно есть своя роль (профиль группы доступа) — набор Use Case почти напрямую превращается в состав прав роли.

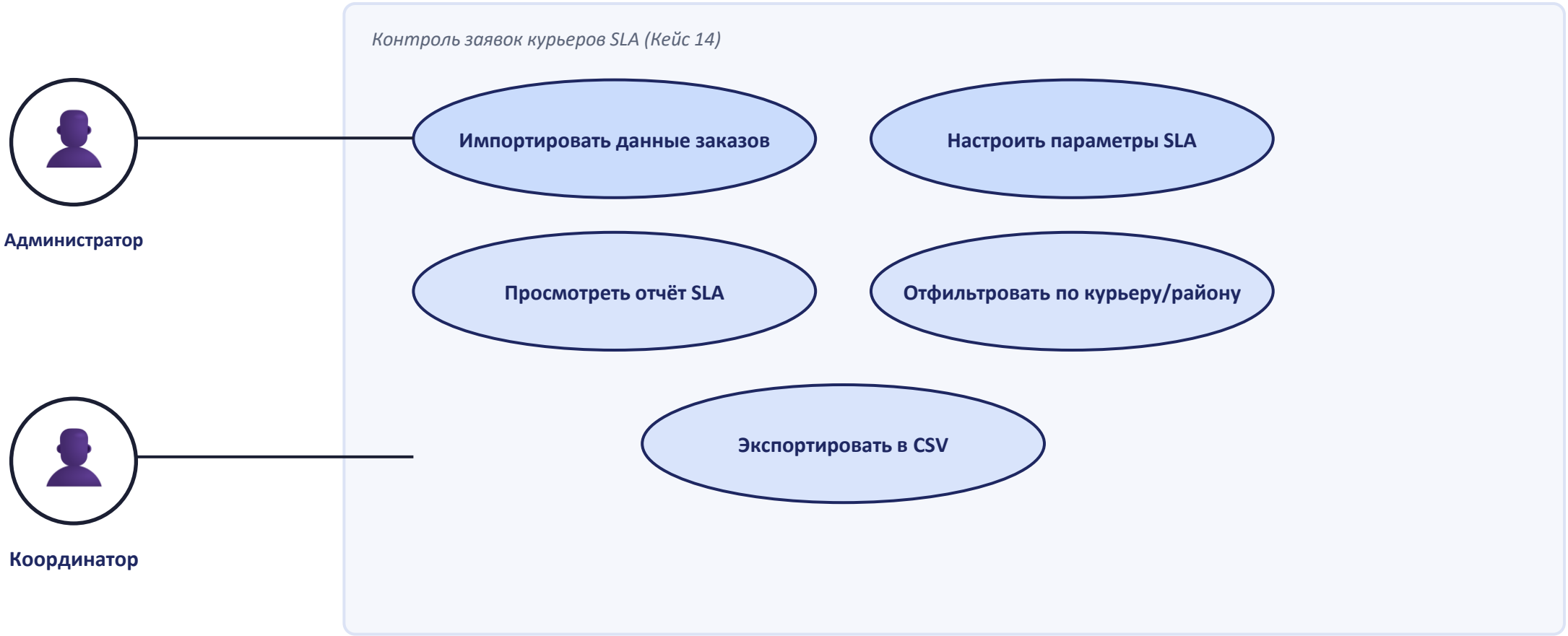
ЗАНЯТИЕ 3

Анатомия пользовательского сценария

1	**Предусловие**	что должно быть истинно до начала сценария (например, менеджер авторизован под ролью «Продажи»)
2	**Основной поток**	шаги «идеального пути» от начала до успешного результата
3	**Альтернативный поток**	отклонение от основного пути, которое всё ещё ведёт к результату
4	**Исключение**	ситуация, при которой сценарий прерывается без результата
5	**Постусловие**	что становится истинным после успешного завершения

ПРИМЕР

Use Case Diagram: Кейс 14 «Контроль SLA»



Для наглядности показаны связи с ключевыми сценариями каждого актёра — в своей диаграмме соедините актёра со всеми применимыми Use Case.

ПРИМЕР

Пример сценария: «Импортировать данные заказов»

Предусловие: администратор авторизован под ролью «Администратор», подготовлены CSV-файлы orders/couriers/trips/statuses

1. Администратор открывает раздел «Импорт данных» и выбирает CSV-файлы
2. Система проверяет статусный поток и ссылочную целостность (order_id, courier_id, trip_id)
3. Система проверяет привязку заказа к рейсу по времени и валидность окон доставки
4. Система загружает данные и пересчитывает метрики SLA (breach_share, avg_delay_minutes)
5. Система показывает протокол загрузки: сколько строк загружено и сколько нарушений SLA найдено

Альтернативный поток: если файл содержит записи, уже загруженные ранее (повторный импорт), — система обнаруживает дубли по order_id и не создаёт повторных записей; в протоколе отмечается число пропущенных дублей.

Постусловие: данные загружены без дублей, метрики SLA пересчитаны, доступен протокол загрузки

Как проверить полноту Use Case

1. У каждого актора из vision есть хотя бы один Use Case?
2. У каждого Use Case явно указан хотя бы один альтернативный поток или исключение?
3. Каждый Use Case приводит к результату, ценному именно для актора, а не для системы?
4. Вторичные акторы (внешние системы) отражены там, где они реально участвуют?

Типичные ошибки в Use Case

Технические детали в сценарии

«система делает UPDATE таблицы statuses» вместо «координатор видит долю нарушений SLA» — Use Case не про реализацию

Актор без цели

«администратор» без пояснения, зачем ему сценарий и какой результат он получает

Только основной поток

без альтернатив и исключений сценарий не готов к реальному использованию

ЗАНЯТИЕ 3

Задача: Use Case для своего продукта

1	**Выпишите актёров**	из vision — целевая аудитория и внешние системы из границ продукта
2	**Назовите 3–5 Use Case**	по одному на ключевую цель каждого первичного актёра
3	**Опишите один сценарий полностью**	предусловие, основной поток, минимум одна альтернатива, постусловие
4	**Проверьте по чек-листу**	со слайда 12 — все ли актёры покрыты хотя бы одним Use Case

ЗАНЯТИЕ 4

Context Diagram: что это

Context Diagram показывает систему как единый прямоугольник, её пользователей и внешние системы, с которыми она взаимодействует — без единой детали о внутреннем устройстве.

ЗАНЯТИЕ 4

Как определить границы системы

Главный вопрос для каждого элемента:

«Эта часть разрабатывается и развивается нашей командой, или это готовая внешняя система, с которой мы просто интегрируемся?»

Наша команда контролирует код → внутри границы.

Готовый сторонний сервис, за который отвечает кто-то другой → снаружи, отдельным прямоугольником.

ЗАНЯТИЕ 4

Нотация Context Diagram

1 Актор (person)

кружок или фигура человека — тот, кто пользуется системой

2 Наша система

один прямоугольник в центре — то, что мы проектируем

3 Внешняя система

прямоугольник другого цвета — готовый сторонний сервис

4 Подписанная связь

стрелка с пояснением: что передаётся и по какому протоколу — HTTP-сервис, веб-сервис, обмен EnterpriseData

От Use Case к Context Diagram

Первичные	**→ Persons**	**Вторичные**	**→ Внешние**
актёры	на диаграмме	актёры	системы

Каждый актор, найденный на занятии 3, уже готов стать элементом контекстной диаграммы — искать заново ничего не нужно.

ПРИМЕР

Context Diagram: Кейс 14 «Контроль SLA»



Как отразить акторов и внешние системы

1. Каждый первичный актор из Use Case показан на диаграмме отдельным элементом?
2. Каждая внешняя система, упомянутая в сценариях (учётная система 1С, моб. приложение курьера, BI-система), нарисована снаружи границы?
3. У каждой связи подписано, что именно передаётся, а не только направление стрелки?
4. Внутри границы — только один прямоугольник нашей системы, без внутренних деталей?

Типичные ошибки в Context Diagram

✕

Внутренние детали на контексте

показаны контейнеры или компоненты — это уже следующий уровень C4

✕

Неподписанные связи

просто стрелка без пояснения — «HTTP-сервис» или «обмен EnterpriseData» важно указать явно

✕

Пропущенная внешняя система

мобильное приложение курьера упомянуто как источник статусов, но забыто на диаграмме

ЗАНЯТИЕ 4

Задача: Context Diagram для своего продукта

1	**Возьмите актёров с занятия 3**	первичные становятся persons на диаграмме
2	**Выпишите внешние системы**	всё, что упоминалось в сценариях, но не разрабатывается вашей командой
3	**Нарисуйте один прямоугольник**	ваша система — без единой внутренней детали
4	**Подпишите все связи**	что передаётся и, если известно, по какому протоколу
5	**Проверьте по чек-листу**	со слайда 20 — ничего не забыто и не показано лишнего

Что сдать по итогам двух занятий

- ✓ Занятие 3: список акторов с указанием первичный/вторичный
- ✓ Занятие 3: 3–5 Use Case и один полностью описанный сценарий с альтернативой
- ✓ Занятие 4: контекстная диаграмма C4 с подписанными связями
- ✓ Занятие 4: все акторы и внешние системы из Use Case отражены на диаграмме

Что дальше

ПРАКТИКИ 5–6

C4 Container Diagram → C4 Component Diagram

Раскроем прямоугольник нашей системы изнутри: сначала на уровне независимо развёртываемых контейнеров — например, толстый клиент, веб-клиент, кластер серверов 1С и внешние HTTP-сервисы, — затем на уровне компонентов внутри одного контейнера.

Принесите на занятие: контекстную диаграмму и список акторов — они станут основой для диаграммы контейнеров.